

[Dashboard](#) / [Meine Kurse](#) / [2023S365114/246/247/248](#) / [Concluding Remarks](#) / [Exam](#)**Begonnen am** Monday, 26. June 2023, 13:56**Status** Beendet**Beendet am** Monday, 26. June 2023, 14:11**Verbrauchte Zeit** 14 Minuten 57 Sekunden**Frage 1**

Richtig

Erreichbare Punkte: 1,00

What is the best possible description for the output of the following snippet of code?

```
@torch.no_grad()
def compute_loss(network: nn.Module, loss_func: nn.Module, x: torch.Tensor, y: torch.Tensor):
    logits = network(x)
    err = loss_func(logits, y)
    err.backward()
    return err.detach()
```

Wählen Sie eine Antwort:

- ☐ a. A Python `list` object with gradients
- ☐ b. A Python `list` object without gradients
- ☐ c. A `torch.Tensor` object with gradients
- ☐ d. A Python `float` object without gradients
- ☒ e. A `torch.Tensor` object without gradients ✓

Frage 2

Falsch

Erreichbare Punkte: 1,00

What is the most accurate description for the following snippet of code?

```
net = nn.Sequential()
net.add_module("features", FeatureNetwork(...))
net.add_module("classifier", nn.Linear(...))

state_dict = torch.load(...)
net.load_state_dict(state_dict)

net.features.requires_grad_(False)
net.classifier.requires_grad_(False)
```

Wählen Sie eine Antwort:

- ☐ a. Preparation of a network with pre-trained features for transfer learning (without feature fine-tuning).
- ☐ b. Preparation of a network with pre-trained features for transfer learning with feature fine-tuning.
- ☒ c. Preparation of a fully pre-trained network for transfer learning (without feature fine-tuning). ✗
- ☐ d. Preparation of a fully pre-trained network for transfer learning with feature fine-tuning.
- ☐ e. Preparation of a network that can not be used for transfer learning

Frage 3

Richtig

Erreichbare Punkte: 2,00

Which *two* of the following concepts do **not** belong in the list of answer options? Concepts are said to belong together if they fit within one of the assignments you made this semester.

Wählen Sie eine oder mehrere Antworten:

- ☐ a. reparameterisation trick
- ☐ b. transposed convolutions
- ☒ c. invertible modules ✓
- ☐ d. `model.decoder(model.encoder(x))`
- ☒ e. fine-tuning ✓

Frage 4

Falsch

Erreichbare Punkte: 1,00

Which of the following lines of code implements the *gradient* computation for projected gradient ascent, which is used to compute adversarial examples?

Wählen Sie eine Antwort:

- ☒ a. `delta += 0.1 * delta.grad.sign()` ✗
- ☐ b. `err.backward()`
- ☐ c. `delta.clamp_(-0.1, 0.1)`
- ☐ d. `delta = torch.zeros_like(x, requires_grad=True)`
- ☐ e. `logits = network(x + delta)`

Frage 5

Richtig

Erreichbare Punkte: 1,00

The code snippet below aims to produce the **objective to be minimised** for the generator of a GAN. You can assume the label for real (fake) labels to be one (zero). What is wrong with this code?

```
def generator_error(gan: nn.ModuleDict, x: torch.Tensor):  
    bce = nn.BCEWithLogitsLoss()  
    z = torch.randn_like(x)  
    with torch.no_grad():  
        fakes = gan.generator(z)  
    logits = gan.discriminator(fakes)  
    fake_labels = torch.zeros_like(logits)  
    err = bce(logits, fake_labels)  
    return -err
```

Wählen Sie eine Antwort:

- ☐ a. The generator error is computed with the wrong kind of loss function.
- ☐ b. The generator weight updates will have the wrong sign.
- ☐ c. The generator and discriminator have been swapped.
- ☒ d. The generator will be unable to receive weight updates. ✓
- ☐ e. The generator will always produce the same outputs during training.

Frage 6

Richtig

Erreichbare Punkte: 1,00

How would you implement the determinant (not the log-determinant) of an additive coupling layer?

☒ a.

```
def det(x):  
    return torch.ones_like(x)
```

 ✓

☐ b.

```
def det(x):  
    return -torch.sum(torch.log(self.conditioner(x[len(x) // 2:])))
```

☐ c.

```
def det(x):  
    return -torch.ones_like(x)
```

☐ d.

```
def det(x):  
    return torch.prod(self.conditioner(x[len(x) // 2:]))
```

☐ e.

```
def det(x):  
    return torch.zeros_like(x)
```

Frage 7

Teilweise richtig

Erreichbare Punkte: 3,00

Match the code snippets to the corresponding ML research papers.

Hint: Try to find algorithms, figures or equations in the papers, there should be no need to read them.

Click on the selected ML research paper to view it:

- **Generative Adversarial Networks**
- **Wasserstein Generative Adversarial Networks**
- **Improved Training of Wasserstein GANs**

```
import torch
from torch import nn, optim
from torch.utils.data import DataLoader
```

```
def generator_error(gan: nn.Module,
                    real_x: torch.Tensor, loss_func:
                    nn.Module):
    ...
```

```
def discriminator_error(gan: nn.Module,
                        real_x: torch.Tensor, loss_func:
```

Improved Training of Wasserstein GANsGenerative Adversarial Networks



```
import torch
from torch import nn, optim
from torch.utils.data import DataLoader
```

```
def generator_error(gan: nn.Module,
                    real_x: torch.Tensor, loss_func:
                    nn.Module):
    ...
```

```
def discriminator_error(gan: nn.Module,
                        real_x: torch.Tensor, loss_func:
```

Generative Adversarial Networks



```
import torch
from torch import nn, optim
from torch.utils.data import DataLoader
```

```
def generator_error(gan: nn.Module,
                    real_x: torch.Tensor, loss_func:
                    nn.Module):
    ...
```

```
def discriminator_error(gan: nn.Module,
                        real_x: torch.Tensor, loss_func:
```

Wasserstein Generative Adversarial Networks

[◀ Final Question](#)

Direkt zu: